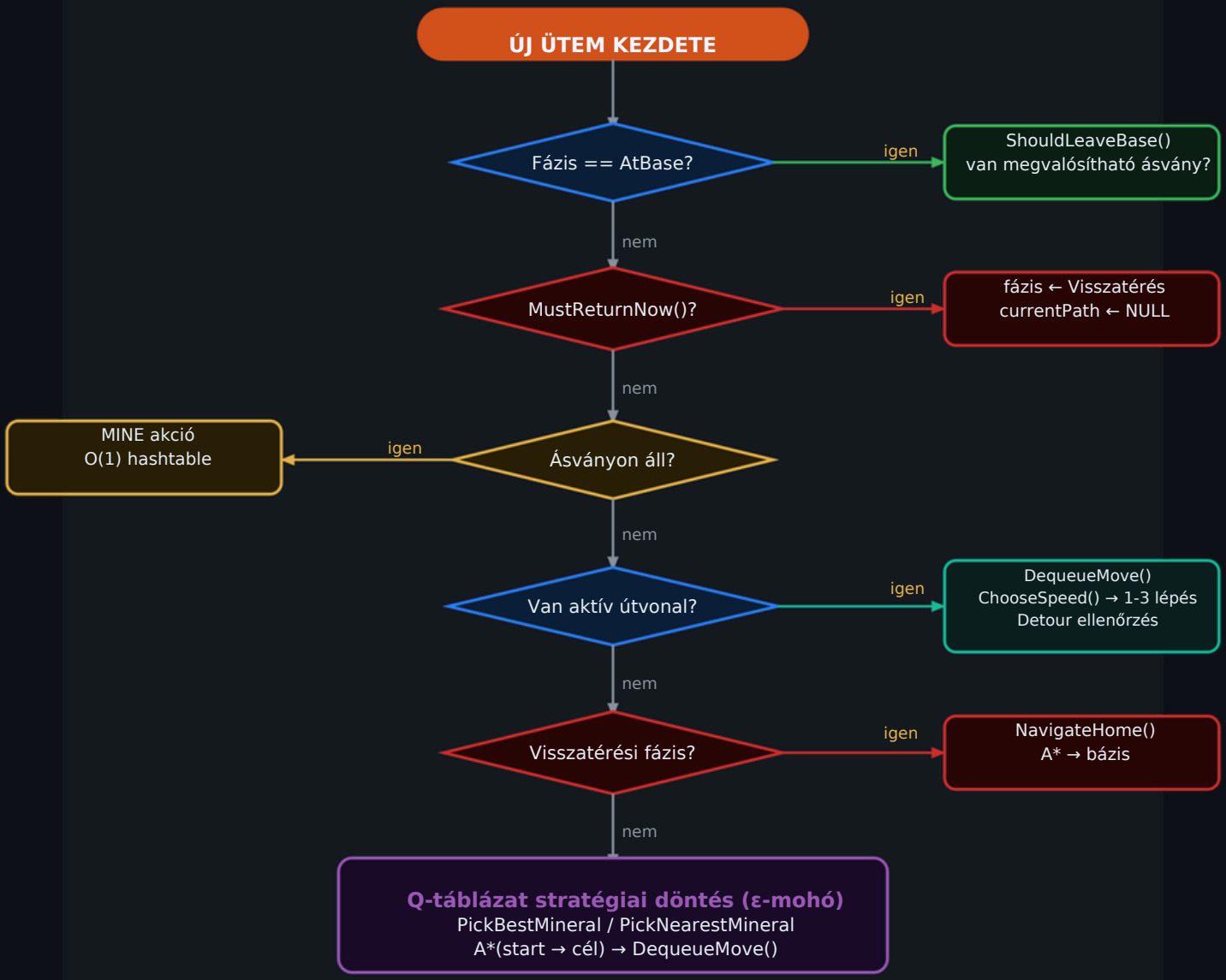
Mars Rover — Hibrid Q-Learning + A*

1. SelectAction(), avagy mit dönt a rover minden fél órában?

Ez az egész rendszer szíve. Minden félórás ütemben pontosan egyszer fut le, és szigorú prioritási sorrendben. Ez dönti el, mit csináljon a rover. Ha egy magasabb prioritású feltétel teljesül, az alacsonyabb prioritásúak már nem futnak le.

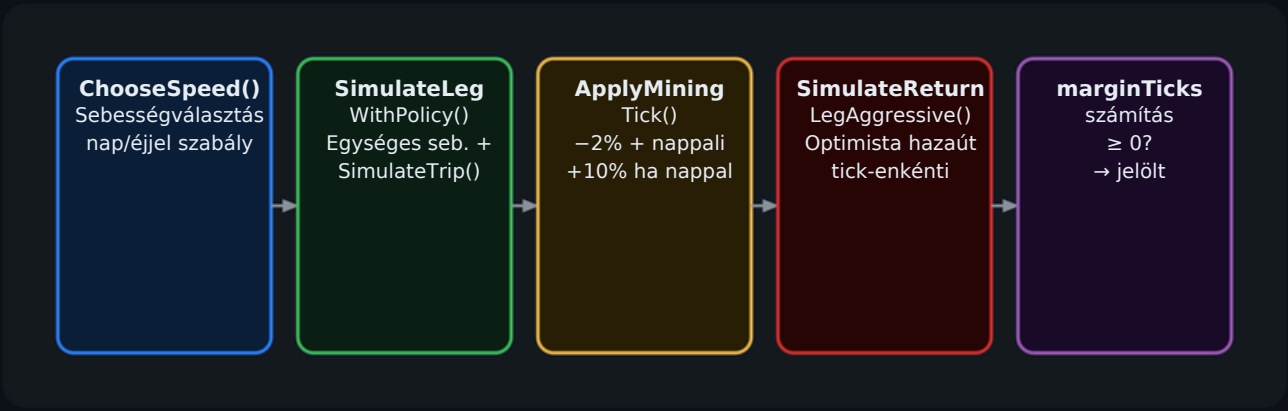


Miért ez a sorrend? A MustReturnNow() biztonsági kapu minden más döntés elé kerül, a rover fizikailag soha nem ragadhat el energiahiány miatt. A Q-táblázat csak akkor szólal meg, ha nincs aktív útvonal és nincs szükséghelyzet, tehát minden más esetben determinisztikus logika irányít.

FindDetourMineral(): Visszatérési fázisban is keres útba eső ásványt (maximum +35% kerülővel). Ha talál, letér, begyűjti, majd folytatja a hazautat.

2. Energia-szimuláció, avagy biztonságos-e odamenni?

Mielőtt a rover elindulna bármelyik ásványhoz, végigszimulál egy teljes körútat. Ha bármelyik ponton az akkumulátor 3% alá süllyedne, az ásvány kiesik a jelöltek közül, nem számít, milyen közel van.

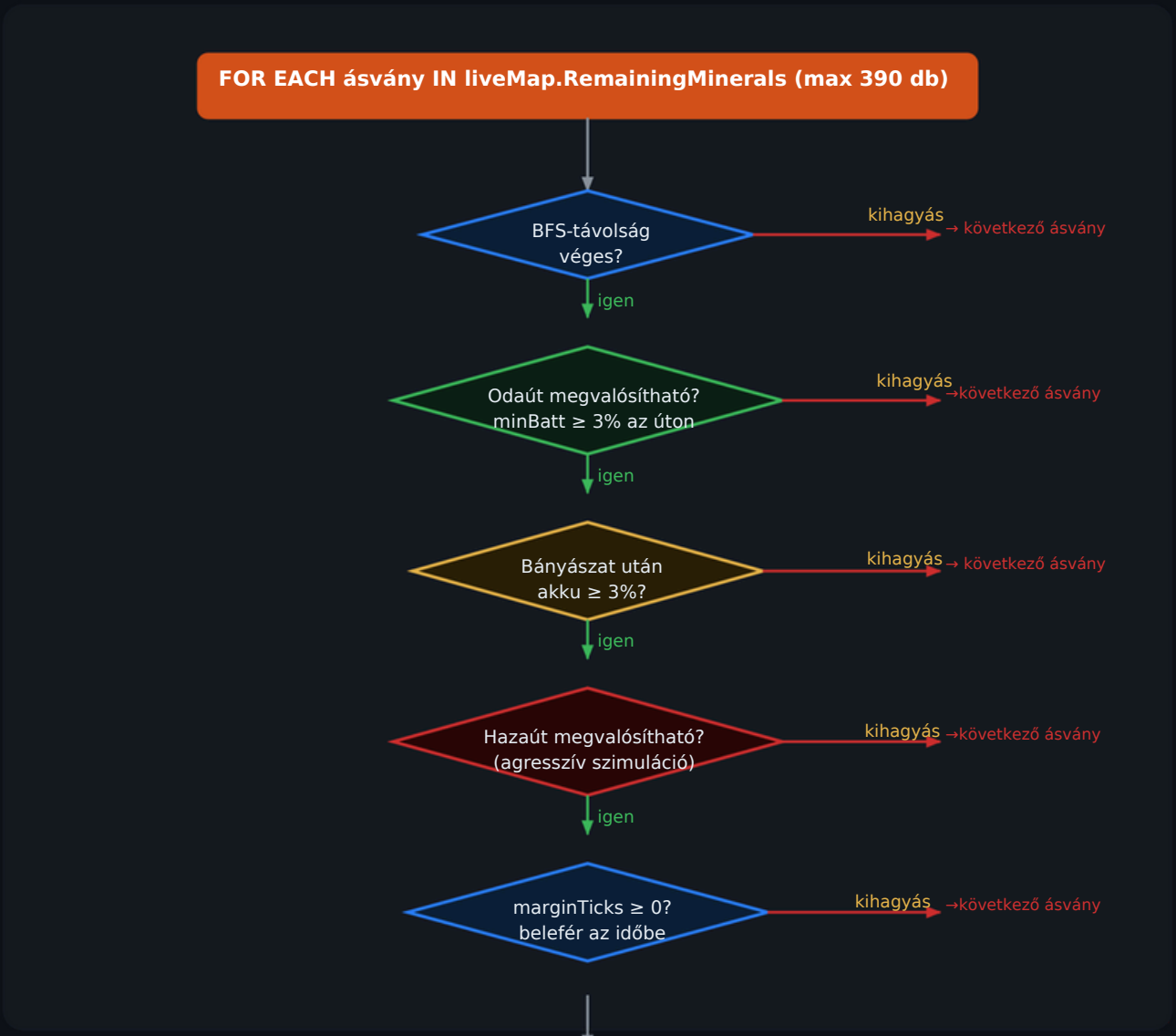


ChooseSpeed(): Nappal a lehető leggyorsabb sebességet választja. Éjszaka külön szabály: 2 lépésnél Normál, 3+ lépésnél Lassú.

SimulateLegWithPolicy(): Egységes sebességet rendel az úthoz, majd EnergyCalculator.SimulateTrip() számol. Nem tick-enkénti. A visszatérési optimista becslés (SimulateReturnLegAggressive) az, ami ütemenként lép.

3. EvaluateFeasibleMinerals() + PickBestMineral()

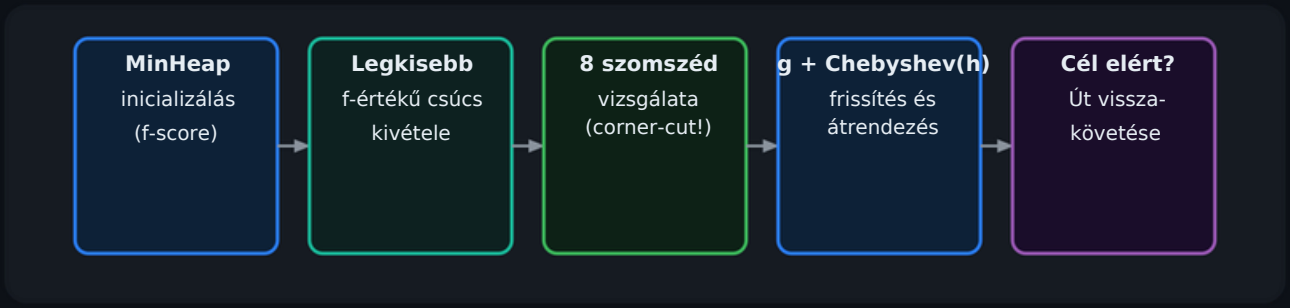
Ez az ötszűrős folyamat fut le mind a legfeljebb 390 ásványra. Csak azok kerülnek a jelöltek listájára, amelyekre a teljes körút, tehát (oda, bányászás, vissza) energetikailag és időben belefér.



Pontszám kiszámítása: 100/(lépések száma + 10) + 1/(lépéskockázat * 2) + jutalom a megtett távolság alapján * 0.75 - megtett idő * 0.0625. Jutalmazza az időtöltést, amíg a helyzet rendezése zajlik. Figyelembe veszi a mineral jutalmazást és bünteti a hosszú hazautat. Így nemcsak a legközelebbi, hanem a legjobb kompromisszumot találja meg. Top-6 lookahead bónusz: A legjobb 6 jelölt esetén megvizsgálja, mi a legjobb elérhető második ásvány az első begyűjtése után. Ez megakadályozza, hogy a rover egy elszigetelt ásványt válasszon egy elérhető klaszter helyett.

4. A* Navigáció, avagy hogyan jut el a Rover a célhoz?

Miután a rover kiválasztotta a célt, az A* megkeresi a legrövidebb akadálymentes utat. MinHeap-et használ, ezért mindig a legígéretesebb irányba halad, nem pazarol időt reménytelen cellákra.



Chebyshev-heurisztika: h(n) = max(|Δx|, |Δy|). Soha nem becsüli túl a valódi távolságot, ezért az A* garantáltan optimális utat talál.	Sarokvágás-tilalom: Átlós lépésnél mindkét szomszéd cellának szabadnak kell lennie. A rover nem súrolhat akadályok sarkát.

5. Q-táblázat, avagy mit tanul meg a rover?

A rover nem mindig ugyanazt a stratégiát alkalmazza, néha a legjobb pontszámú ásványt keresi, néha a legközelebbit. A Q-táblázat tanulja meg, melyik helyzetben melyik a nyerő.

A 6. dimenzió: ReturnMarginBucket (4 szint): Mennyi idő marad még hazaérni? ≤0 (kritikus), 1–2, 3–5, 6+ ütem. Ez megakadályozza, hogy a rover túl sokáig maradjon kint.

A 7. dimenzió: BatteryTrendBucket (3 szint): Az akkumulátor csökken, stabil, vagy nő? Ez mutatja, hogy a rover veszteséges vagy nyereséges pályán van-e.
A korábbi 6D, 1 800 állapotos változat ezeket nem tudta kezelni.

6. Tanítási hurok, avagy hogyan tanul a rover 1 000 epizódon át?

A rover próbálgat, hibázik és tanul. Az első epizódokban szinte teljesen véletlenszerűen cselekszik (ε=1.0), majd a tapasztalat felhalmozódásával egyre inkább azt csinálja, amit jónak tud (ε=0.05).

Mechanizmus	Mit csinál?	Kulcsparaméter
TD(λ) frissítés	A TD-hibát visszaterjeszti a korábbi döntésekre is nem csak az utolsó lépés tanul belőle, hanem a megelőző 3-4 lépés is kap kreditet.	λ = 0.7 γ = 0.95

Eligibility trace	Nyilvántartja a nemrég meglátogatott állapotokat. 3 ütemmel korábbi döntés a TD-hiba 0.7^3 = 0.343-szorosát kapja meg frissítéskor.	e(s,a) += 1 minden lépésnél γλ csökkentés
Prioritized Experience Replay	Ritka de kritikus szituációkat (pl. alacsony akku, határeset döntések) felülreprezentálja a mintavételben, így gyorsabban tanulja meg a nehéz eseteket.	ω = 0.6 β: 0.4 → 1.0 fontossági súly
ε-greedy policy	Felfedezés és kihasználás egyensúlya: ε valószínűséggel véletlen akciót választ (felfedezés), egyébként a legjobb ismert Q-értékű döntést (kihasználás).	ε: 1.0 → 0.05 exponenciális csökkentés